A pattern framework driven by the Hamming distance for structured light-based reconstruction with a single image

Xavier MAURICE, Pierre GRAEBLING and Christophe DOIGNON
LSIIT (UMR CNRS 7005), Strasbourg University, France
xavier.maurice@lsiit-cnrs.unistra.fr , http://lsiit-cnrs.unistra.fr/avr-fr

Abstract

Structured light based patterns provide a means to capture the state of an object shape. However it may be inefficient when the object is freely moving, when its surface contains high curvature parts or in out of depth of field situations. For image-based robotic guidance in unstructured and dynamic environment, only one shot is required for capturing the shape of a moving region-of-interest. Then robust patterns and real-time capabilities must be targeted. To this end, we have developed a novel technique for the generation of coded patterns directly driven by the Hamming distance. The counterpart is the big amount of codes the coding/decoding algorithms have to face with a high desired Hamming distance. We show that the mean Hamming distance is a useful criterion for driving the patterns generation process and we give a way to predict its value. Furthermore, to ensure local uniqueness of codewords with consideration of many incomplete ones, the Perfect Map theory is involved. Then, we describe a pseudorandom/exhaustive algorithm to build patterns with more than 200×200 *features, in a very short time, thanks to a splitting strategy which performs the Hamming tests in the codeword space instead of the pattern array. This leads to a significant reduction of the computational complexity and it may be applied to other purposes. Finally, real-time reconstructions from single images are reported and results are compared to the best known which are outperformed in many cases.*

1. Introduction

Structured light (SL) is a well-known means to bring simple cues onto an unknown scene. In conjunction with camera devices, depth recovery or local reconstruction of poorly textured surfaces can be achieved with feature-based geometrical techniques, and in the past SL has been widely used for capturing the shape of static (or motion controlled) objects with stripes, grey or color encoding ([15], [8], [19], [24], [4], [11]). To deal with free moving objects, [22] with phase-shift and stereo and [23] with stripe lines provided shape reconstruction techniques with single shot. In [10], dense shape reconstruction is made possible thanks to the use of local connection information of a grid pattern. Last year, Sagawa *et al.* also proposed fast dense 3D reconstruction from a single image by means of de Bruijn sequences coded lines and belief propagation [18].

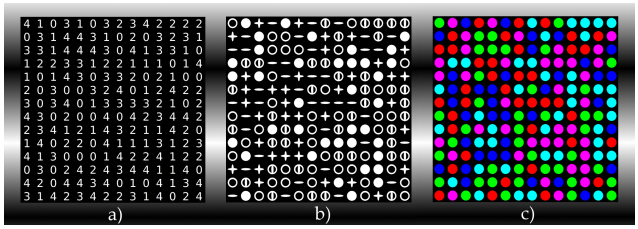


Figure 1. A 5-ary, 14×14 Perfect Submap (PSM)-based patterns with a Hamming distance of 5 between all 3×3 neighbourhoods, represented with a) digits, b) shapes, c) colors. At least 2 errors per neighbourhood can be corrected.

To match visual features in the image with those of the pattern components, a coding strategy is performed. Codes are associated to visual features to get unique identification with spatial or temporal structures. To achieve both a reliable identification and accurate location, several approaches have been proposed and most of them have been summed up in [3] and classified in three main categories in [20]. For our application of concern, we are looking for approaches based on spatial neighbourhood. Indeed, time-multiplexing approaches do not allow reconstruction of moving scenes and direct coding is too sensitive to illumination changes, more than spatial neighbourhood which is rather useful to discriminate symbols by geometry through a sophisticated coding. With colors, shapes or grey intensities (see Fig. 1), one gets a huge number of window-based neighbourhood combinations (the codes) which increases the feature discrimination power, yielding patterns robust to noise, out-of-focus or highly curved surfaces as reported in ([1],[12]).
One may observe during several years that to get more accurate reconstruction with SL, efforts are concentrated on

Authorized licensed use limited to: Lakehead University. Downloaded on September 29,2026 at 16:10:10 UTC from IEEE Xplore. Restrictions apply.

1	2	3	4	5	6
[8] **Griffin**	PM	NA	11×29	3	spots
[14] **Morano**	PSM	1 to 4^*	45×45	3	spots
[20] **Salvi**	Grid of slits (De Bruijn)	NA	372	6	cross-points
[16] **Pages**	Hybrid (De Bruijn)	NA	128	4 (hue levels)	lines and peaks
[2] **Albitar**	PSM	6 (average)	27×29	3	disk, ring, segment
[5] **Claes**	PSM	3	64×85	6	spots (hexag. window)
[1] **Adán**	PSM	3 (average)	986	7	spots (hexag. window)
our	PSM and SPSM	1 to 5^*(PSM),6^*(SPSM)	150×100 (SPSM)	4	cuneiforms

Table 1. Pseudo-random encoded grids and M-arrays techniques: 1.reference; 2.pattern type (PM: Perfect Maps; PSM: Perfect Submaps; SPSM: Subperfect Submaps); 3.Hamming distance (*: min); 4.max pattern size; 5.number of symbols (max); 6.associated features.

the pattern coding. Then, the objective of this work is to provide fast construction and decoding schemes for such patterns with big size and embedding high data redundancy with the Hamming distance as the main criterion.

1.1. Terminology

The Hamming distance between two codewords is the number of positions where symbols differ. If not specified elsewhere, the following notations are used: H_{Min}: minimum Hamming distance; $\bar{H}$: mean Hamming distance; C: alphabet size (number of symbols); W: codewords length; L: size of a L^2 square pattern.

1.2. A brief review on close related works

As a preamble, we recall that a matrix P is a C-ary $(m, n; u, v)$ Perfect Map (PM) if it is a $m \times n$ array in which all the C-ary $u \times v$ submatrices appear exactly once. PM is the 2-D version of De Bruijn sequences [6] which have been widely applied ([13], [17], [19]). To get high resolution, Pagès [16] used an hybrid pattern also based on De Bruijn sequences composed of a stripe-pattern coded in the RGB space and grey intensities coded in a peak-based pattern. Direct pattern constructions based on PM proposed by [7] and [9] were restricted to the binary case ($C = 2$). Furthermore they did not allow to choose the pattern size which would be suitable for high resolutions, nor to impose a Hamming distance greater than 1 for embedding correction capabilities.

A more flexible approach is to look for brute-force algorithms, such iterative algorithms fills the C-ary pattern of a desired size by trial and error, checking the suited Hamming distance at each iteration. Morano [14] and more recently Albitar [2] with geometrical features and Claes [5] with colour spots proposed such codification, and since not all the submatrices are used, therefore they were named Perfect Submaps (PSM) and Adan *et al.* [1] used PSM to design colour coded hexagonal arrays.

The present work is closely related to previous PSM-based approaches. Here, to achieve accurate real-time reconstructions, very big patterns are generated. The global correctness power can be computed in advance thanks to the mean Hamming distance and the pattern generation is controlled via the minimum Hamming distance. Short time pattern generation and real-time decoding are also provided (see Table 1). To realize such improvements, the unicity tests are performed in the codeword space and in a close neighbourhood, then P is named Subperfect Submap (SPSM).

1.3. Paper outline

In section 2, the usefulness of the mean Hamming distance as a new design parameter is presented. In section 3 the complexity reduction for brute-force algorithms based on PSM is studied (3.1) and we present a new algorithm based on SPSM, driven by the desired minimum Hamming distance with direct access to the codeword space (3.3). With these contributions, patterns bigger than those previously reported for the same parameters set can be provided in a very short time as discussed in section 4.

2. Fundamentals for pattern design

In this section, we aim at bringing some understandings of the pattern statistics prior to the pattern generation.

2.1. Pattern design and the mean Hamming distance

We reconsider the widely spread idea that given the pattern size, window size and H_{Min}, using fewer symbols is always better. We show that a little bit more symbols statistically increases the correction capabilities a lot, in a degressive way.

Let us consider the typical Hamming distribution for several C values and with $(W, H_{Min}) = (9, 1)$, as shown in Fig. 2. One can see that when C increases, the number of code couples that has a small H value decreases. This means that it is more probable to decode all couples with a small H with a higher C value. For example, in Fig. 2, we can observe that if $C = 8$, it is more probable that all code couples with $H \geq 5$ have all been uniquely decoded than with $C = 3$, since with $C = 8$ the number of such couples is drastically lower than with $C = 3$. If these couples are decoded, then, all other codes couples should have a $H > 5$, allowing error corrections, despite the fact that the desired H_{Min} of the pattern was only set to 1. So, statistically, the case $C = 8$

Authorized licensed use limited to: Lakehead University. Downloaded on September 29,2026 at 16:10:10 UTC from IEEE Xplore. Restrictions apply.

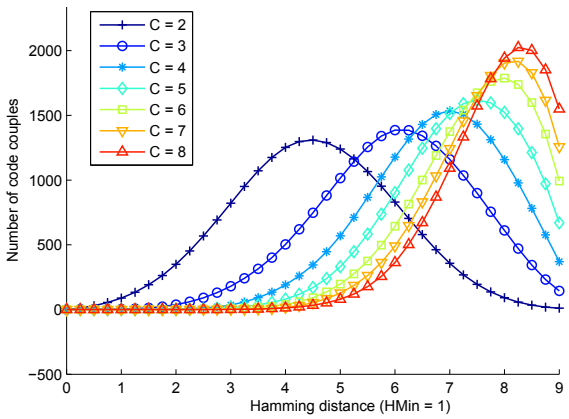


Figure 2. Smoothed Hamming distance distributions for patterns with C symbols, $H_{Min} = 1$ and $W = 9$ (mean over 100 pseudo-randomly PSM-based patterns of same size, for several C values).

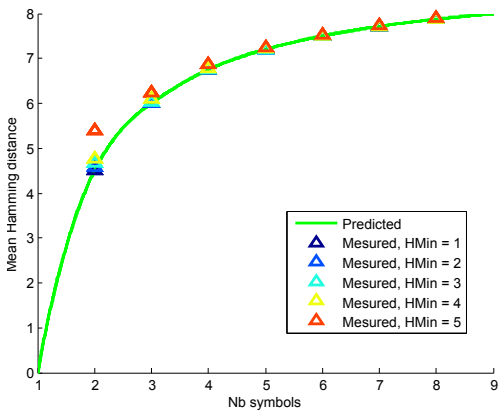


Figure 3. Estimated $\bar{H}$ for $W = 9$ vs the number of symbols (in green) and the computed $\bar{H}$ for several patterns (mean over 100 patterns for each H_{Min}) with a growing desired H_{Min}.

allows more corrections than the case $C = 3$. The higher C, the less important the gain ratio is between two successive C values (4 symbols instead of 3 yields higher benefits than the correction gain with 8 symbols instead of 7).
We conclude that using a little bit more symbols, up to a value that does not induce significant symbols discrimination troubles, can yield interesting correction gains. It could be interesting to know this statistical correction gain evolution in advance. To do so, the behaviour of the Hamming distance distributions is assumed to be well-represented by the mean values evolution in the sequel.

2.1.1 Definition of the mean Hamming distance $\bar{H}$

Let S be a set of N codewords with length W. The $\bar{H}$ of S is defined as the sum of Hamming distances between all couples of code words divided by the number of couples:

$$\bar{H} = \frac{\sum_{i=1}^{N} \sum_{j=i+1}^{N} \text{Ham}(C_i, C_j)}{N(N-1)/2} \tag{1}$$

where $\text{Ham}(C_i, C_j)$ is the Hamming distance between the two codes C_i and C_j, i.e. the number r of positions (or indices in the code string), several symbols differ

$$\text{Ham}(C_i, C_j) = \sum_{r=1}^{W} \delta_r \;\; ; \delta_r = \begin{cases} 0 & \text{if } C_{ir} = C_{jr} \\ 1 & \text{otherwise.} \end{cases} \tag{2}$$

2.1.2 A priori estimation of $\bar{H}$

During perfect maps/submaps codification, no symbol should be favoured. Therefore, assuming the C symbols are uniformly distributed in the codewords, algorithms in accordance with this fully benefit of the $\bar{H}$ analysis.

Proposition : Assuming a uniform distribution of symbols across codewords of length W chosen in a C-ary alphabet, the mean Hamming distance $\bar{H}$ between any couples of codewords is given by $\bar{H} = W - W/C$.

Proof: Let $A = \{A_1, ..., A_W\}$ and $B = \{B_1, ..., B_W\}$ two code words of length W, where $A_i, B_i \in \{0, 1, ..., C-1\}$. A_i and B_i can take any of the C values with an equiprobability of $1/C$. Consequently, the probability the same symbol occurs in A and B at the ith position is expressed by

$$\begin{array}{rcl} p(A_i = B_i) & = & p(A_i = 0) \times p(B_i = 0) \\ & + & \ldots \\ & + & p(A_i = C-1) \times p(B_i = C-1) \\ & = & \sum_{j=0}^{C-1} p(A_i = C_j) \times p(B_i = C_j) \\ & = & \sum_{j=0}^{C-1} (1/C) \times (1/C) = 1/C. \end{array} \tag{3}$$

Hence the expectation E of the total number of identical symbols between A and B is

$$\begin{array}{rcl} E & = & p(A_1 = B_1) + \ldots + p(A_W = B_W) \\ & = & \sum_{i=1}^{W} p(A_i = B_i) = W/C \; . \end{array} \tag{4}$$

Finally, by complementarity, it comes $\bar{H} = (W - E)$.

As shown in Fig. 3, the above $\bar{H}$ function is very close to values measured a posteriori with many generated patterns using (1). Only the case $(C, H_{Min}) = (2, 5)$ presents a bias, but this case is rarely used in practice. From these results, the $\bar{H}$ evolution wrt C is very similar to the expected correction gain previously discussed, i.e. once again the gains are larger when successive C values are lower.

3. Pattern generation

In this section we present a new brute-force-*like* pattern generation algorithm with a (3×3) window property, which

Authorized licensed use limited to: Lakehead University. Downloaded on September 29,2026 at 16:10:10 UTC from IEEE Xplore. Restrictions apply.

is a good tradeoff between the codewords length and pattern sizes, and which allows comparisons with other works.

3.1. Computational complexity reduction

During the pattern generation process, brute-force approaches execute tests on new elements step by step to check the desired H_{Min}. If N codes have been already included, each new code created with the new element has to be compared to these N codes to check whether the H_{Min} constraint holds. So the algorithmic complexity of the H_{Min} tests is $O(N)$. We provide a way to reduce this complexity in the case $H_{Min} = 1$ in subsection 3.3.1, and for higher H_{Min} values in 3.3.2.

3.2. Previous subperfect submaps algorithms

We briefly present the algorithms proposed by Morano in [14] (**M**) and by Claes in [5] (**C**) to easily tackle the new one hereafter. In (**M**), successive pseudo-random elements are added to the pattern. The pattern is filled raw by raw from the top left corner. If the new element contributes to a novel (3×3) submatrix, the H_{Min} constraint of the associated code is checked with all already included codes. If the test is passed, the next element is processed, otherwise the $C - 1$ other digits/symbols are tried. If no digit gives a positive result, the incomplete pattern is cancelled, and the process is restarted.
In (**C**), a full exhaustive search is performed using clever rules to test only promising branches of the search tree. All symbols are initially set to 0. When the H_{Min} test is negative, previously found digits are converted to a single but large C-ary number which is always incremented in C-base to test new configurations. As all the digits of this number evolve from 0 to the $C-1$ value, all configurations are tested. If the algorithm tries to increment the first digit from $C-1$ to C, then the search space is exhausted. Thus, one is sure that if a solution exists for a given parameters set, it will be found, which is not the case with [14]. Moreover, in (**C**) the patterns are only tested once.

Limits of these approaches: In (**M**), the strategy is purely random but this approach does not capitalize much on the already found incomplete solutions, the search is being too early cancelled. In (**C**), the strategy is purely exhaustive: But as soon as the matrix size and the symbols number become significant, the combinatory explosion implies that only a small part of the solution space can be tested in a reasonable time (several hours of computations may be needed for large patterns !). Moreover, as the most significant digits are rarely incremented, the 0 symbol is over represented and so the distribution is not homogeneous at all.

3.3. Proposed algorithm

A common task in all previous approaches is the unicity test related to the Hamming distance. As far as we know, this test is performed in the array space (the pseudo-random array that is being built). Indeed, whatever the search behaviour of the algorithm (random, exhaustive, with a correction windows) and whatever the set of codes between which one wants the H_{Min} constraint to hold, the Hamming distance between each new code to test and codes in the array has to be computed. So independently to implementation optimisations it is of practical importance to reduce more the number of required tests so as to deliver big pattern in few seconds only. The number of Hamming distance tests to be performed to check for H_{Min} constraint with a new code is named the H *test complexity*.
To better tackle the pattern process, we report the following bound due to [21] from the theory of information field. That is a lower bound on the number C of symbols required to create N codewords of length W with a minimum H_{Min} Hamming distance is $C \geq N^{1/(W-(H_{Min}-1))}$. It follows that the space of C-ary codewords of length W, named $\Omega_{C,W}$, is such that $\text{card}(\Omega_{C,W}) = C^{(W-(H_{Min}-1))}$.

The algorithm implements the exploration/exploitation paradigm well known in the artificial intelligence field and presented in [12]. It can be seen as a generalization of the two preceding approaches, a trade-off strategy between that of [14], which explores the solution space purely randomly and that of [5], purely exhaustive. However, the key idea here is to perform the H *test* in the codewords space.
If a $m \times n$ pseudo-random array P has to be coded, we can reasonably state that a lower bound on the test complexity is $\mathcal{O}((m \times n) \div 2)$ for each new code. With the same objective as that of the work of [12] which takes advantage of the known epipolar geometry in order to reduce this complexity to a one-dimensional space only, here we describe a method to decrease the H *test complexity* while being able to produce either PSM or SPSM, whatever the relative configuration of the underlying devices.

3.3.1 Complexity reduction for $H_{Min} = 1$

Each code can only be used once. The unicity for all possible codewords can be represented by a boolean flag array with one cell per codeword. The codewords space C_S is exploited in the following manner: in the 1-D array containing C^W binary flags, each flag is uniquely adressed by a codeword of $\Omega_{C,W}$ such that the W symbols are the coefficients in C basis (e.g. with $C = 3$, $W = 9$, the 000000012 codeword corresponds to the decimal address $2 \times 3^0 + 1 \times 3^1 + ... + 0 \times 3^8 = 5$). The general formula is $\text{Address}(A) = \sum_{i=1}^{W} A_i \, C^{W-i}$.
If a flag is set to *False* then the corresponding codeword

Authorized licensed use limited to: Lakehead University. Downloaded on September 29,2026 at 16:10:10 UTC from IEEE Xplore. Restrictions apply.

is not yet used, otherwise it is. At the beginning, all the flags are set to *False*. Then for each new code A to test, all the flags corresponding to codewords with $H < H_{Min}$ wrt A are checked. Then if all these flags are set to *False*, the H_{Min} test is true, A can be included in P and its corresponding flag is set to *True*, otherwise a new code must be tried. Let us consider the trivial case where $H_{Min} = 1$. Then for any W and C, A has to be checked only with itself ($H = (H_{Min} - 1) = 0$) in C_S, leading to a H *test complexity* of $\mathcal{O}(1)$, ie, a direct access vs more than $(m \times n) \div 2) = 5000$ tests for a 100×100 pattern with tests in the array space. With a pre-computed array, the code flag is directly accessible, hence the unicity test is made in $O(1)$ instead of $O(N)$. This makes a serious advantage when the size of patterns becomes very large.

Memory requirements: as flags for all possible codewords need to be stored, C^W boolean flags are necessary and so C^W bits. For the range considered in our study (Fig. 7), this is equivalent to 8^9 Bits $= 16$ MBytes only.

3.3.2 Generalisation to other H_{Min} values

For any C, W and H_{Min}, the number of flags N_f to be scanned in the codeword space C_S is

$$N_f(H_{Min}) = 1 + \sum_{h=1}^{H_{Min}-1} \frac{W!}{h!(W-h)!} (C-1)^h \,. \quad (5)$$

Proof *(and algorithm)*: For each $h \in [0, H_{Min}[$, the number of codes (N_C) with a hamming distance of h with codeword A is $N_C = N_W \times (C-1)^h$, where $N_W = \frac{W!}{h!(W-h)!}$ is the number of combinations for choosing h elements among W, without taking the order into account. These are all the possible sets of h positions of the symbols that are not the same as in codeword A between the W possible. The factor $(C-1)^h$ is, for each set of h positions, the number of possible symbols combinations if the symbols at the h positions are not the same as in A. Indeed, for each position, there are $(C-1)$ possible different symbols wrt that initially in A at this position and therefore $(C-1)^h$ symbols combinations for h positions. The first term is accounting for $h = 0$.

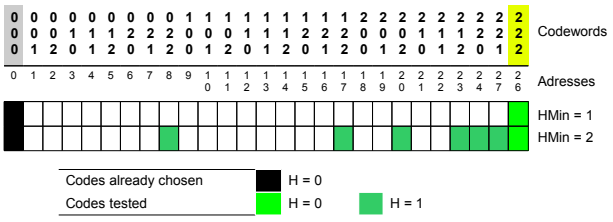


Figure 4. The H_{Min} tests performed in the array of codes adress.

This is illustrated in Fig.4 with $C = W = 3$ and for $H_{Min} = 2$ (last lane). In this example, the new codeword to test is $\{2,2,2\}$ with adress$(\{2,2,2\}) = 26$. The number of tests to perform is $N_f = 1 + \frac{3!}{(3-1)!} \cdot (3-1)^1 = 1+6 = 7$ in the 1-D adress array to ensure a $H_{Min} = 2$ with all other codewords.The corresponding 7 cells have been drawn in green ($h = 0$ in light green (the codeword itself!) and $h = 1$ in dark green). Then, all the other addresses are corresponding to codewords with a Hamming distance ≥ 2 with $\{2,2,2\}$ and are drawn in white and black (code $\{0,0,0\}$ is assumed to be already included in the pattern).

The splitting strategy: The previous method is efficient for small H_{Min} values, as the index h is an exponent number in the above formula (5). For high H_{Min} values, we rather propose to split this exponent (thus the index h) by means of intermediate codewords. These codewords are chosen so as to capitalize the tests already done with selected codewords. To tackle this, it is worth to notice that given two codewords A_1 and A_2 with a Hamming distance of $H_{1,2}$, there exists a third codeword A_3 with respectively the H_1 and H_2 Hamming distances from A_1 and A_2 and such that $H_{1,2} = H_1 + H_2$ with $H_1 \leq H_{1,2}$ and $H_2 \leq H_{1,2}$. Up today, the algorithm implements a unique intermediate codeword, the number of tests is then

$$N_f(H_{Min}) = N_f(H_1) + N_f(H_2) \quad (6)$$

with $H_1 = \text{floor}((H_{Min}-1)/2)$ and $H_2 = H_{Min}-1-H_1$.

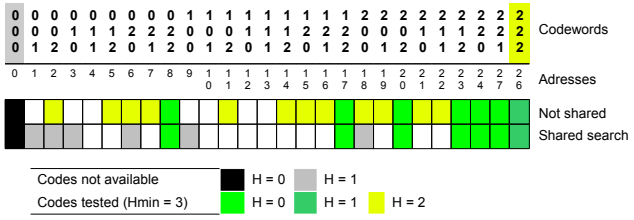


Figure 5. The H_{Min} tests performed wit two subsets of the codes adress array, with an intermediate codeword.

4. Experimental results

In Fig. 5, we consider again the previous example but with $H_{Min} = 3$. The number of tests is then $N_f(3) = 1+7+\frac{3!}{(2!(3-2)!} \cdot (3-1)^2 = 19$, the latter is corresponding to yellow and green cells (first lane).With the splitting of Hamming distances, codewords $\{0,0,0\}$ and $\{2,2,2\}$ both need to look only for codes having $h = (0,1)$, instead of just $\{2,2,2\}$ to look for codes having $h = (0,1,2)$. As the number of tests grows exponentially with h (formula (5), this strategy requires less tests. The number of tests is simply

Authorized licensed use limited to: Lakehead University. Downloaded on September 29,2026 at 16:10:10 UTC from IEEE Xplore. Restrictions apply.

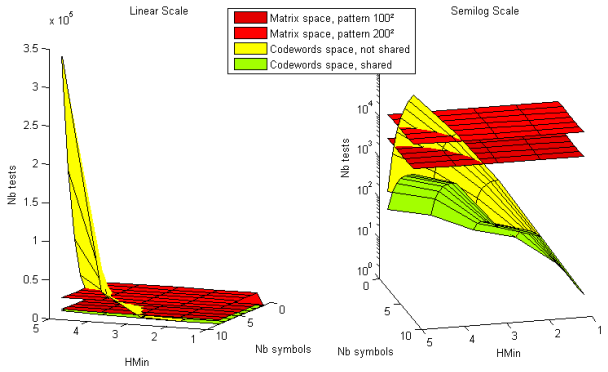


Figure 6. Estimated number of unicity tests using search in the matrix space, codewords space with and without a shared search.

the addition of $N_f(2) = 7$ (black and grey cells) with those of the applicant codeword $\{2, 2, 2\}$ with also $N_f(2) = 7$, the latter is corresponding to the green cells (last lane). This yields 14 tests vs 19. This gain is far more demonstrative with parameters values of typical patterns (Fig. 6). Despite the exponential complexity w.r.t h, the search in the codewords space with a splitting strategy (Fig.6,Green) requires far less tests than typical search in the matrix space. This is due to the very low dynamics of h in the shared strategy.

4.1. Perfect submaps coding

Perfect submaps coding results for $H_A = W_A = 3$, $H_{Min} \in [1, 5]$, a (3×3) window property, and $C \in [2, 8]$ are reported on Fig. 7. These parameters allow comparisons with those reported by Claes [5] (**C**) and Morano [14] (**M**). With the available resources (Pentium 4 3Ghz, 2Go RAM), we get the best results ("Our Max" on Fig.7) in a few minutes with $H_C = W_C = 5$. In all cases our algorithm gives larger matrix than (**M**), who tried to generate matrices of size 45^2. In average, our algorithm needs 2 symbols less. When $H_{Min} > 2$ our approach behaves similarly to (**C**) in terms of size. With $H_{Min} \leq 2$, our algorithm clearly makes the difference. Indeed, a too exhaustive behaviour leads to a combinatory explosion for (**C**) while our algorithm keeps on finding PSM with dimensions growing quasi linearly with C. Moreover, for a low HMin.
Timing considerations show the other main advantage of the proposed approach: it can generate patterns of significant sizes quite online. In Figure 7, orange columns ("Our $\leq$1s"), we reported the maximum pattern size we could obtain in less than one second. Although we did not implement parallel or SIMD optimizations yet, we could generate in one second patterns with sizes greater than those usually used with similar techniques (grids and M-Arrays as reported in [1]), where the mean patterns size was around 20^2 and 64×63 for the maximum. Other approaches did not report timing results, but (**C**) needed hours to find its maximum while we needed few seconds only by doing the unicity tests in the codewords space. Only the splitted search approach 3.3.2 is reported. As expected, the non splitted met complexity explosions with H_{Min} values higher than $2, 3$. All maximum results were produced after few seconds on a Core 2 Duo standard PC with 2 Go RAM. ***Other noticeable results:*** patterns of very important sizes have been found : 9620^2 for $H_{Min} = 1$ and 1645^2 for $H_{Min} = 2$ with only 8 symbols for low $H_{Min}values$, where the search in the codewords space is the most efficient. The ever increasing resolutions in the upcoming decades will allow to take advantage of these sizes. The 22^2 pattern for $H_{Min} = 1$ and $C = 2$ uses 400 codes, which is 78% of the maximum (512). Big patterns can be be generated very fast, this is very praticle when one needs to try different patterns to find the most adapted to a target application : size, robustness or the number of used symbols can be priviledged. Moreover such a speed to generate usable patterns opens new perspective for online adaptive patterns. Some would want to change the number of symbols online or the resolution.

4.2. 3-D Recontructions

We illustrate the advantages of such an efficient coding with a set of 3-D reconstructions generated with a monochromatic pattern using 4 cuneiform shapes (Fig.8(a,b)). These are oriented along epipolar lines to avoid the need for rotational invariance and to ease the segmentation process. Also the unicity of codes is not ensured between all the possible couples of codes but only between those which have the central feature (of their 3×3 neighbourhoods), on the same epipolar lines (SubPerfect SubMaps), as suggested by [12]. This results in patterns with higher correction capabilities than Perfect Submaps. In all the presented reconstructions a pattern of dimensions 100×150, with 4 symbols and $H_{Min} = 4$ was used. Figure 9 exhibits the "one-shot" property of such patterns and its ability to capture moving objects, here a moving hand. The fingers and the occlusions they produce can be accurately captured thanks to the efficient coding using local clues only (3×3) primtives to enable the matching process. In Figure 10 general geometrical and spectral robustness of produced pattern is assessed. One can observe the gain in capture quality by using a high resolution pattern (ours) Fig. 11.a vs patterns using typical M-array resolution (50×50) Fig.11.b which can not properly capture the fingers. Figure .12.a illustrates the capture of thin strings of a summer shoe and finally Figure 12.b shows that such high resolution patterns can enable markerless motion captures : the limbs and their occlusions on the body can be properly handled.

5. Conclusion

We have proposed a new framework for the design of patterns for reconstruction purposes through active vision

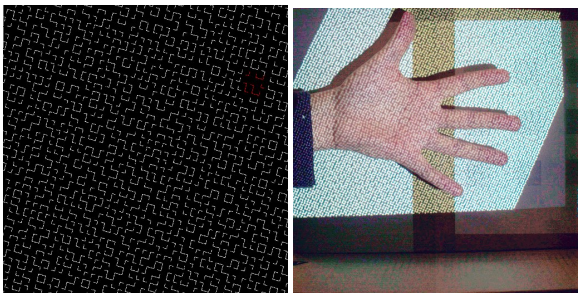
Authorized licensed use limited to: Lakehead University. Downloaded on September 29,2026 at 16:10:10 UTC from IEEE Xplore. Restrictions apply.

k/H_{Min}	1				2			3				4			5		
Note					*Claes* : N.A.							*Claes* : N.A.			*Morano* : N.A.		
Algo	*Our <1s*	*Our Max*	*C.*	*M.*	*Our <1s*	*Our Max*	*M.*	*Our <1s*	*Our Max*	*C.*	*M.*	*Our <1s*	*Our Max*	*M.*	*Our <1*	*Our Max*	*C.*
2	20^2	22^2	19x25	NA	10^2	11^2	NA	6^2	7^2	6x8	NA	5^2	5^2	NA	NA	NA	4x5
3	109^2	111^2	106x114	$>= 45^2$	34^2	37^2	$< 45^2$	15^2	16^2	14x18	$< 45^2$	9^2	9^2	$< 45^2$	6^2	7^2	6x8
4	397^2	418^2	316x421	$>= 45^2$	103^2	104^2	$< 45^2$	34^2	37^2	33x44	$< 45^2$	16^2	17^2	$< 45^2$	10^2	10^2	9x11
5	815^2	1155^2	429x572	$>= 45^2$	242^2	262^2	$>= 45^2$	67^2	73^2	68x90	$< 45^2$	27^2	27^2	$< 45^2$	12^2	14^2	12x16
6	855^2	2580^2	496x661	$>= 45^2$	429^2	548^2	$>= 45^2$	123^2	133^2	124x165	$< 45^2$	41^2	43^2	$< 45^2$	17^2	20^2	18x23
7	905^2	5205^2	676x901	$>= 45^2$	445^2	1023^2	$>= 45^2$	196^2	228^2	204x272	$< 45^2$	44^2	64^2	$< 45^2$	19^2	27^2	24x32
8	905^2	9620^2	696x925	$>= 45^2$	365^2	1645^2	$>= 45^2$	223^2	344^2	292x389	$>= 45^2$	22^2	92^2	$< 45^2$	14^2	35^2	32x42

Biggest pattern, size increase : < 100% or N.A. | 100% - 500% | > 500%

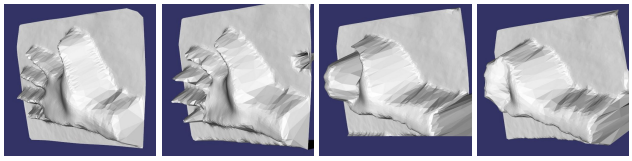
Biggest pattern we generate in less than 1 sec :

Figure 7. Comparative results of perfect submaps generation. N.A. : Not Avail., M. : [Morano98] C. : [Claes08].



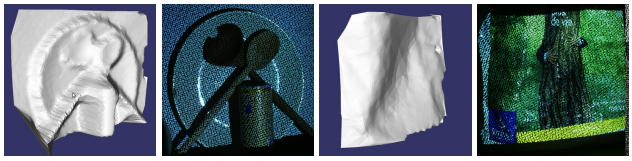
(a) (b)

Figure 8. (a) Projection of a 100×150 pattern, with $H_{Min} = 4$ and $C = 4$ symbols (associated to cuneiforms). (b) Pattern projected on a hand scene



(a) (b) (c) (d)

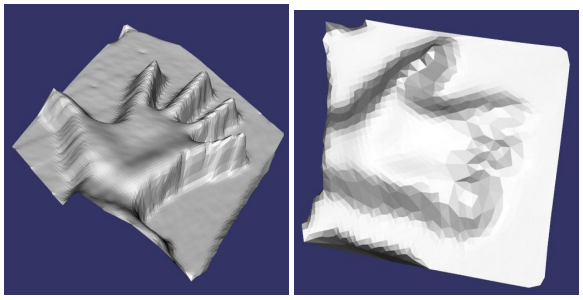
Figure 9. Reconstruction of a real-time closing hand : (a to d).



(a) (b) (c) (d)

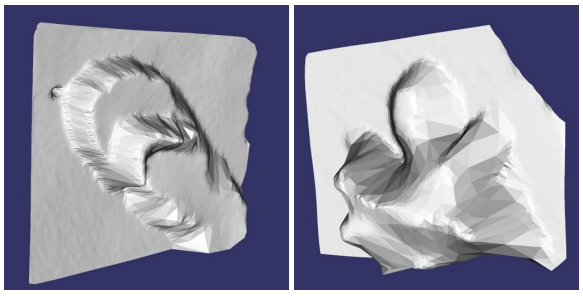
Figure 10. Robustness of the method w.r.t geometrical difficulties : (a,b) occlusions by thin objects and spectral difficulties (c,d)

with coded structured light. To achieve accurate reconstructions despite disturbances commonly encountered while



(a) (b)

Figure 11. (a) High resolution vs (b) typical resolution hand capture.



(a) (b)

Figure 12. (a) Summer shoe with its thin strings captured. (b) A full human bust with crossed arms.

dealing with unstructured and changing environments, very big patterns with high correction capabilities highlighted during the coding/decoding stages have been provided. The mean and min Hamming distances have been studied. It has been shown that the former brings the overall pattern statistics about correctness power and can be computed prior to the pattern generation. The latter has been used both to drive the pattern generation process and also to conduct optimi-

Authorized licensed use limited to: Lakehead University. Downloaded on September 29,2026 at 16:10:10 UTC from IEEE Xplore. Restrictions apply.

sations by performing the numerous tests in the 1-D codewords adress space instead of the 2-D pattern array. The splitting strategy developed to carry out those tests leads to a significant reduction of the computational complexity. It is the main pratical assessment for delivering 200×200-size patterns (that today sensors can handle) in only few seconds and also for providing real-time reconstructions by means of CUDA implementations for parallel executions.
We plan to use this framework to generate very big patterns for intra-abdominal real-time reconstruction of organs' surface through endoscopic device. This will be useful for assisting surgeons with augmented reality tools and also for tracking robotized surgical instruments.

Acknowledgements
This work is part of the MIX-Surg IHU project and has been made possible thanks to supports of the Region Alsace Council, the french Ministery of Research and the Karl Storz company.

References

[1] A. Adán, A. Vázquez, C. Cerrada, and S.Salamanca. Moving surface extraction based on unordered hexagonal perfect submaps projection: Applications to 3d feature tracking. *Image Vision Computing*, 27(8):1083–1096, 2009. 2497, 2498, 2502

[2] C. Albitar, P. Graebling, and C. Doignon. Robust structured light coding for 3d reconstruction. In *Int'l Conf. on Computer Vision*, Rio de J., Brazil, 2007. 2498

[3] J. Batlle, E. Mouaddib, and J. Salvi. Recent progress in coded structured light as a technique to solve the correspondence problem: a survey. *Pattern Recognition*, 31(7):963–982, 1998. 2497

[4] S. Chen, Y. Li, and J. Zhang. Realtime structured light vision with the principle of unique color codes. In *IEEE Int'l Conf. on Robotics and Automation*, pages 429–434, April 2007. 2497

[5] K. Claes. *Structured Light Adapted to Control a Robot Arm*. PhD thesis, Kath. Univ. Leuven, 2008. 2498, 2500, 2502

[6] N. G. de Bruijn. A combinatorial problem. In *Nederlanske Akad. van Wetenschappen*, pages 758–764, 1946. 2498

[7] T. Etzion. Constructions for perfect maps and pseudo-random arrays. *IEEE Int'l Trans. on Information Theory*, 34(5/1):1308–1316, 1988. 2498

[8] P. M. Griffin, L. S. Narasimhan, and S. R. Yee. Generation of uniquely encoded light patterns for range data acquisition. *Pattern Rec.*, 25(6):609–616, 1992. 2497, 2498

[9] G. Hurlbert and G. Isaak. On the de bruijn torus problem. *J. Combin. Theory Ser. A*, 64:50–62, 1993. 2498

[10] H. Kawasaki, R. Furukawa, R. Sagawa, and Y. Yagi. Dynamic scene shape reconstruction using a single structured light pattern. In *Int'l Conf. on Computer Vision and Pattern Recognition*, Anchorage, 2008. 2497

[11] T. Koninckx and L. V. Gool. Real-time range acquisition by adaptive structured light. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 28(3):432–445, March 2006. 2497

[12] X. Maurice, P. Graebling, and C. Doignon. Real-time structured light coding for epipolar-dependent patterns. In *SPIE Photonics Europe*, volume 7724, Brussels, Belgium, April 2010. 2497, 2500, 2502

[13] C. J. Mitchell. Aperiodic and semi-periodic perfect maps. *IEEE Transactions on Information Theory*, 41(1):88–95, 1995. 2498

[14] A. Morano, C. Ozturk, R. Conn, S. Dubin, S. Zietz, and J. Nissanov. Structured light using pseudo-random codes. *IEEE Int'l Trans. on Pattern Analysis and Machine Intelligence*, 20(3):322–327, 1998. 2498, 2500, 2502

[15] H. Morita, K. Yajima, and S. Sakata. Reconstruction of surfaces of 3-d objects by m-array pattern projection method. *ICCV*, 88:468–473, 1988. 2497

[16] J. Pagès, J. Salvi, C. Collewet, and J. Forest. Optimised de bruijn patterns for one-shot acquisition. *Image and Vision Computing*, 23:707–720, 2005. 2498

[17] K. G. Patterson. Perfect maps. *IEEE Int'l Trans. on Information Theory*, 40(3):743–753, 1994. 2498

[18] R. Sagawa, Y. Ota, Y. Yagi, R. Furukawa, and N. Asada. Dense 3d reconstruction method using a single pattern for fast moving object. In *Int'l Conf. Computer Vision*, Kyoto, Japan, 2009. 2497

[19] J. Salvi, J. Batlle, and E. Mouaddib. A robust-coded pattern projection for dynamic 3d scene measurement. *Patern Recognition Letters*, 19:1055–1065, 1998. 2497, 2498

[20] J. Salvi, J. Pagès, and J. Batlle. Pattern codifications strategies in structured light systems. *Pattern Recognition*, 37:827–849, 2004. 2497, 2498

[21] R. Singleton. Maximum distance q-nary codes. *IEEE Trans. on Information Theory*, 10:116–118, 1964. 2500

[22] T. Weise, B. Leibe, and L. V. Gool. Fast 3d scanning with automatic motion compensation. In *Int'l Conf. on Computer Vision and Pattern Recognition*, Minneapolis, MN, USA, June 2007. 2497

[23] M. Young, E. Beeson, J. Davis, S. Rusinkiewicz, and R. Ramamoorthi. View-point coded structured light. In *Int'l Conf. on Computer Vision and Pattern Recognition*, Minneapolis, MN, USA, June 2007. 2497

[24] L. Zhang, B. Curless, and S. Seitz. Rapid shape acquisition using structured light and multi-pass dynamic programming. In *Int'l Symp. on 3D Data Processing, Visualization and Transmission*, pages 24–36, 2002. 2497

Authorized licensed use limited to: Lakehead University. Downloaded on September 29,2026 at 16:10:10 UTC from IEEE Xplore. Restrictions apply.